SIMATS ENGINEERING

(Saveetha School of Engineering)

GIT & DOCKER TECHNICAL ASSIGNMENT

Fraud Detection and Financial Transaction Monitoring System

CO3 Assessment Tool – Git & Docker Technical Assignment

Full-Stack Application with Version Control and Containerization

Assignment Contents

- 1. Problem Analysis
- 2. Project Structure Design
- 3. Git Repository Initialization
- 4. Git Branching Strategy
- 5. Version Control Workflow
- 6. Commit History Analysis
- 7. Docker Environment Design
- 8. Dockerfile Development
- 9. Docker Compose Design
- 10. Container Deployment and Testing
- 11. Benefits of Git and Docker
- 12. Challenges and Improvements
- Architecture and Component Interaction
- Quality Attribute Analysis
- Architectural Justification
- Final Conclusion

1. Problem Analysis

1.1 Problem Statement

Financial institutions process a large number of transactions every day. Some transactions may be fraudulent, suspicious or unusual compared with a customer's normal transaction behaviour. Manually checking every transaction is time-consuming and may delay the identification of fraud.

The proposed Fraud Detection and Financial Transaction Monitoring System monitors financial transactions and identifies potentially suspicious transactions using predefined rules and a fraud detection model. The system provides alerts to authorized users so that suspicious transactions can be reviewed quickly.

1.2 Project Objectives

1. To record and monitor financial transactions.
2. To identify suspicious transactions automatically.
3. To calculate a fraud risk score for each transaction.
4. To generate alerts for high-risk transactions.
5. To allow authorized users to review transaction details.
6. To maintain transaction and alert history.
7. To provide a simple dashboard for monitoring.
8. To improve security, reliability and maintainability through proper software architecture.
9. To use Git for version control and collaborative development.
10. To use Docker for consistent application deployment.

1.3 Functional Requirements

Requirement	Description
User Login	Authorized users can securely log into the system.
Transaction Recording	The system records transaction details.
Transaction Monitoring	Transactions are continuously checked for suspicious behaviour.
Fraud Detection	The fraud service analyses transaction information.
Risk Scoring	Each transaction receives a risk score.

Alert Generation	High-risk transactions generate alerts.
Alert Management	Users can view and update alert status.
Transaction Search	Users can search and filter transactions.
Dashboard	Displays transaction and fraud monitoring information.
Report Generation	The system provides transaction/fraud reports.

1.4 Non-Functional Requirements

- Security: Transaction and user information must be protected.
- Performance: Transactions should be processed within an acceptable response time.
- Scalability: The system should support increasing transaction volumes.
- Reliability: The system should continue operating correctly during normal failures.
- Availability: Monitoring services should remain available when required.
- Maintainability: Individual modules should be easy to modify and test.
- Usability: The dashboard should be simple for authorized users.
- Portability: The application should run consistently using Docker containers.

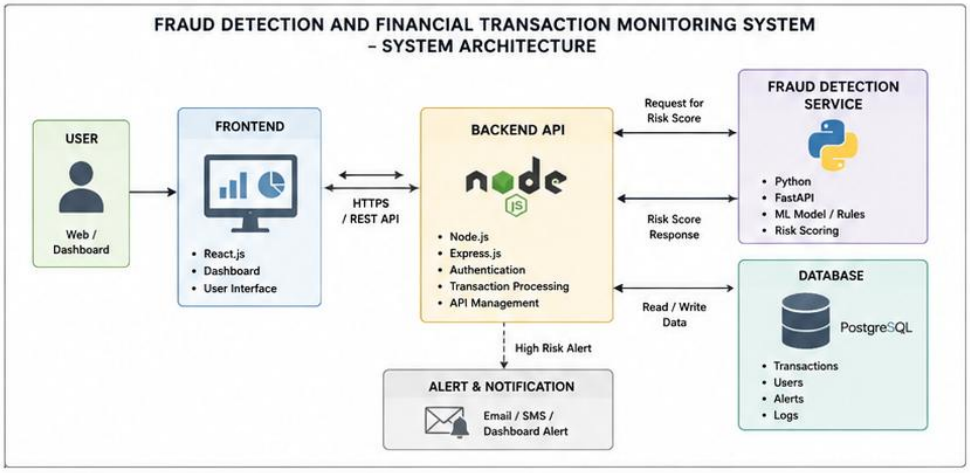
1.5 Expected Outcomes

The expected result is a working system that can accept financial transactions, analyse them, assign a fraud risk level and generate alerts for suspicious activity.

The use of Git provides controlled version management, while Docker provides a consistent environment for development, testing and deployment.

IMAGE 1: SYSTEM ARCHITECTURE DIAGRAM

This diagram shows the overall architecture of the Fraud Detection and Financial Transaction Monitoring System and how each component interacts.



2. Project Structure Design

The repository is organized into separate folders so that each component has a clear responsibility.

```
fraud-monitoring-system/
│
├── frontend/
│   ├── src/
│   ├── public/
│   ├── package.json
│   └── Dockerfile
│
├── backend/
│   ├── src/
│   │   ├── controllers/
│   │   ├── routes/
│   │   ├── services/
│   │   ├── models/
│   │   └── middleware/
│   ├── package.json
│   └── Dockerfile
│
├── fraud-service/
│   ├── app/
│   │   ├── main.py
│   │   ├── model.py
│   │   └── predictor.py
│   ├── requirements.txt
│   └── Dockerfile
│
├── database/
│   ├── init.sql
│   └── seed.sql
│
├── docs/
│   ├── architecture.md
│   ├── api-documentation.md
│   └── screenshots/
│
├── tests/
│   ├── backend/
│   └── fraud-service/
│
├── .gitignore
├── README.md
├── Dockerfile
└── docker-compose.yml
```

Purpose of Each Major Folder

frontend/ Contains the user interface and dashboard.

backend/ Contains REST APIs, authentication, transaction processing and communication with the database.

fraud-service/ Contains the fraud detection logic/model and generates a risk score.

database/ Contains database initialization and sample data.

tests/ Contains test cases for different application components.

docs/ Contains architecture diagrams, API documentation and project screenshots.

3. Git Repository Initialization

3.1 Create the Repository

```
mkdir fraud-monitoring-system
cd fraud-monitoring-system


git init
git branch -M main


git remote add origin https://github.com/username/fraud-monitoring-system.git
```

3.2 Check Git Status

```
git status
```

This displays the files that are untracked, modified or ready to be committed.

3.3 First Commit

```
git add .
git commit -m "chore: initialize fraud monitoring project"
git push -u origin main
```

3.4 Purpose of Essential Git Files

- .git/ – Stores Git repository information and version history.
- .gitignore – Prevents unwanted and sensitive files from being tracked.
- README.md – Explains the project and setup process.
- main branch – Stores stable project code.

4. Git Branching Strategy

Selected Strategy: Main + Develop + Feature Branches

A simplified Git Flow strategy is suitable for this project.

```
main
  |
  └── develop
       ├── feature/login
       ├── feature/transaction-monitoring
       ├── feature/fraud-detection
       ├── feature/dashboard
       └── feature/docker
```

main Contains stable code suitable for release.

develop Contains integrated development code before it is released to main.

feature/* Used to develop individual features without directly modifying the stable branches.

```
git checkout develop
git checkout -b feature/fraud-detection
```

After completing the feature:

```
git add .
git commit -m "feat: add transaction fraud risk scoring"
```

```
git push -u origin feature/fraud-detection
```

Why This Strategy Is Suitable

This strategy separates stable code from active development. It reduces accidental changes to the main branch and allows different features to be developed independently. It also supports code review and easier conflict management.

5. Version Control Workflow

```
Create Issue
     ↓
Create Feature Branch
     ↓
Develop Feature
     ↓
Test Feature
     ↓
git add
     ↓
git commit
     ↓
git push
     ↓
Pull Request
     ↓
Code Review
     ↓
Merge to develop
     ↓
Testing
     ↓
Merge to main
```

Meaningful Commit Examples

```
git commit -m "feat: add transaction monitoring API"
git commit -m "feat: implement fraud risk scoring"
git commit -m "fix: correct transaction validation"
git commit -m "test: add fraud detection test cases"
git commit -m "docs: update Docker deployment instructions"
```

Synchronization

```
git checkout develop
git pull origin develop
```

After completing the work:

```
git add .
git commit -m "feat: add fraud alert dashboard"
git push origin feature/dashboard
```

Merge Conflict Resolution

```
git pull origin develop
```

Git identifies the conflicting files. The developer manually selects the correct changes, then:

```
git add .
git commit -m "fix: resolve merge conflict"
git push
```

This keeps the repository synchronized and reduces the chance of losing another developer's changes.

6. Commit History Analysis

A good commit history should clearly explain how the project developed.

```
9f31abc  docs: update deployment documentation
7c21d45  feat: add Docker Compose configuration
5a72c19  feat: implement fraud risk scoring
4b63e10  feat: add transaction monitoring API
3d52a11  feat: create transaction database model
2a41b08  feat: create login functionality
1a30c07  chore: initialize project
```

Analysis

The history shows the project developing in logical stages. The first commit creates the basic repository. Authentication and transaction functionality are then implemented. Fraud detection is added after the transaction processing layer. Docker configuration is added after the application components are available.

Commit messages use a clear format such as:

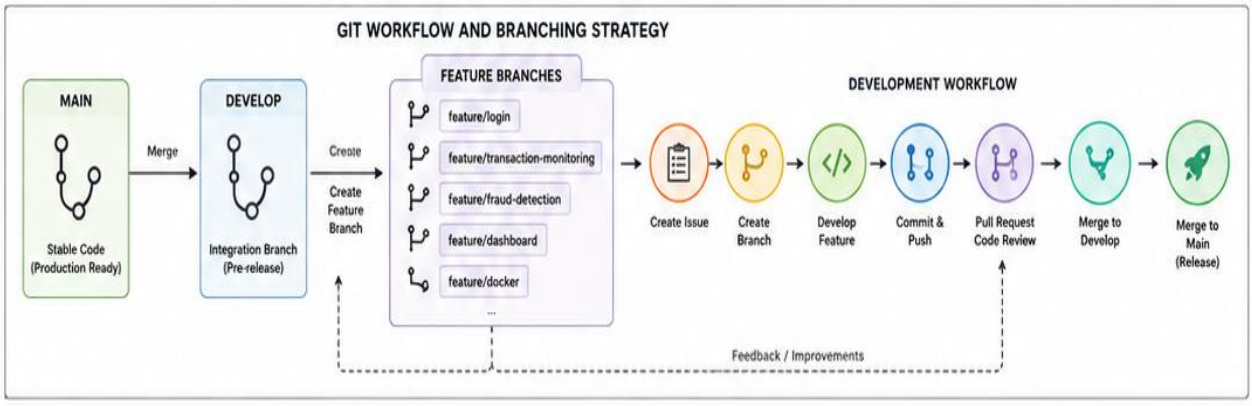
```
feat:
fix:
test:
docs:
chore:
```

Benefits of Good Commit History

- Makes project development easy to track.
- Helps identify when a problem was introduced.
- Makes code review easier.
- Allows previous versions to be restored.
- Improves collaboration between developers.
- Provides evidence of systematic development.

IMAGE 2: GIT WORKFLOW DIAGRAM

This diagram illustrates the Git branching strategy and workflow followed in the project.



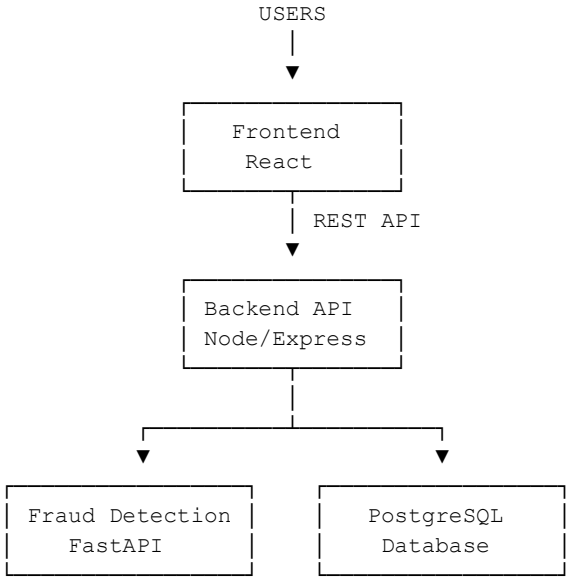
7. Docker Environment Design

Docker is suitable for this project because the system contains multiple components with different dependencies.

Components Requiring Containerization

11. Frontend
12. Backend API
13. Fraud Detection Service
14. PostgreSQL Database

Architecture



Docker Network

All application containers communicate through a private Docker network.

```
fraud-network
     |
     ├── frontend
     ├── backend
     ├── fraud-service
     └── postgres
```

The backend can communicate with PostgreSQL using the service name:

```
postgres:5432
```

8. Dockerfile Development

A Dockerfile defines how an application container is built.

Backend Dockerfile

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./
```

```
RUN npm ci

COPY . .

EXPOSE 5000

CMD ["npm", "start"]
```

Explanation

- FROM selects the base image.
- WORKDIR sets the application directory.
- COPY package*.json copies dependency information.
- RUN npm ci installs dependencies.
- COPY . . copies source code.
- EXPOSE documents the application port.
- CMD starts the backend application.

Fraud Detection Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Important Dockerfile Practices

- Use small base images where appropriate.
- Do not store passwords inside the Dockerfile.
- Use environment variables for configuration.
- Use .dockerignore.
- Install only required dependencies.
- Keep images simple and maintainable.

Example .dockerignore:

```
node_modules
.git
.env
__pycache__
*.log
```

9. Docker Compose Design

Docker Compose is used because the project contains multiple services that must work together.

docker-compose.yml

```
services:
```

```
  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    depends_on:
      - backend
    networks:
      - fraud-network

  backend:
    build: ./backend
    ports:
      - "5000:5000"
    environment:
      DATABASE_URL: postgresql://frauduser:fraudpass@postgres:5432/frauddb
      FRAUD_SERVICE_URL: http://fraud-service:8000
    depends_on:
      - postgres
      - fraud-service
    networks:
      - fraud-network

  fraud-service:
    build: ./fraud-service
    ports:
      - "8000:8000"
    networks:
      - fraud-network

  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: frauduser
      POSTGRES_PASSWORD: fraudpass
      POSTGRES_DB: frauddb
    ports:
      - "5432:5432"
    volumes:
      - postgres-data:/var/lib/postgresql/data
    networks:
      - fraud-network

volumes:
  postgres-data:

networks:
  fraud-network:
```

Services

Service	Purpose	Port
Frontend	User interface	3000
Backend	REST API	5000
Fraud Service	Fraud analysis	8000
PostgreSQL	Data storage	5432

Dependencies

```
Frontend
   ↓
Backend
```

```
    ├──→ Fraud Service
    └──→ PostgreSQL
```

10. Container Deployment and Testing

Build the Containers

```
docker compose build
```

Start the Application

```
docker compose up -d
```

The -d option runs containers in detached mode.

Check Running Containers

```
docker compose ps
NAME                STATUS
frontend            running
backend             running
fraud-service       running
postgres            running
```

View Logs

```
docker compose logs
docker compose logs backend
```

Test the Services

Frontend: Open http://localhost:3000

Backend: Example endpoint: http://localhost:5000/api/health

Fraud Service: Example: http://localhost:8000/docs

Expected backend response:

```
{
  "status": "OK"
}
```

Test Database Connectivity

The backend sends a database request to PostgreSQL. A successful response confirms that Backend → PostgreSQL communication is working.

Stop Containers

```
docker compose down
```

Stop and Remove Volumes

```
docker compose down -v
```

The second command should be used carefully because it removes the database volume and stored database data.

Testing Procedure

15. Start all containers.
16. Check container status.
17. Open the frontend.

18. Log into the system.
19. Submit a test transaction.
20. Backend receives the transaction.
21. Backend sends transaction information to the fraud service.
22. Fraud service calculates the risk score.
23. Backend stores transaction and result in PostgreSQL.
24. Dashboard displays the transaction and alert.
25. Check logs for errors.
26. Stop and restart containers to verify deployment reliability.

11. Benefits of Git and Docker

Benefits of Git

Version Management – Git stores different versions of the project and allows previous versions to be restored.

Collaboration – Multiple developers can work on separate branches without directly modifying stable code.

Traceability – Commit history shows who made a change and what was changed.

Branching – Features can be developed independently and merged after testing.

Backup – A remote Git repository provides a copy of the source code and its history.

Benefits of Docker

Consistency – The same application environment can be used on different computers.

Portability – The application can be moved between development, testing and deployment environments.

Isolation – Each service runs in its own container with its required dependencies.

Scalability – Individual services can be replicated when the system requires higher capacity.

Easy Deployment – The complete application can be started using docker compose up -d.

Maintainability – Services are separated, making it easier to update or replace individual components.

12. Challenges and Improvements

12.1 Challenges Encountered

Dependency Management – Different components require different software versions and dependencies. Solution: Docker containers are used to isolate dependencies.

Container Communication – The frontend, backend, fraud service and database need to communicate correctly. Solution: Docker Compose networking and service names are used.

Database Persistence – Data can be lost if the database container is removed without persistent storage. Solution: A Docker volume is used for PostgreSQL.

Git Merge Conflicts – Changes made by different developers may affect the same files. Solution: Feature branches, regular synchronization and code review are used.

Environment Configuration – Passwords and database connection details should not be stored directly in source code. Solution: Environment variables and .env configuration are used.

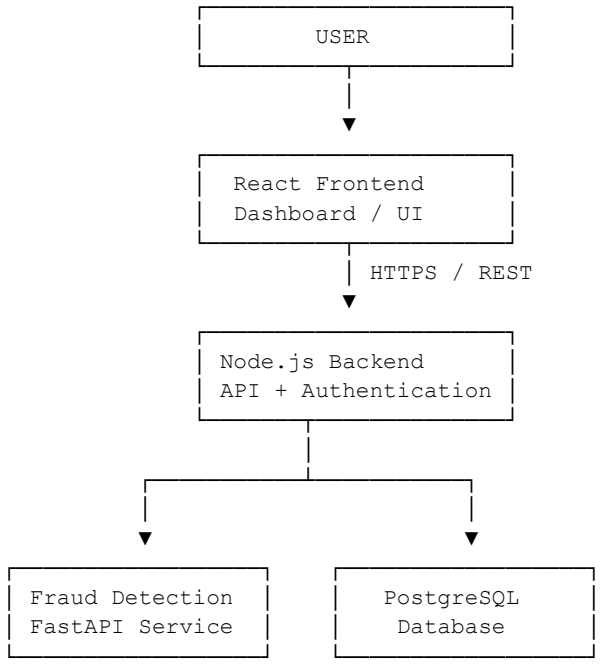
Security – Financial transaction information is sensitive. Solution: Authentication, authorization, encrypted communication, input validation and secure secret management should be implemented.

Future Improvements

27. Add a machine-learning fraud detection model with improved accuracy.
28. Add real-time transaction monitoring.
29. Add email/SMS notification for critical alerts.
30. Add role-based access control.
31. Add automated GitHub Actions CI/CD.
32. Add automated unit and integration testing.
33. Add centralized logging and monitoring.
34. Use Docker secrets or a secure secret-management system.
35. Add database backup and recovery mechanisms.
36. Deploy the containers to a cloud platform.
37. Add load balancing for high transaction volumes.
38. Add an audit trail for important administrative operations.

Architecture and Component Interaction

Overall Architecture



Transaction Workflow

```
User submits transaction
          ↓
Frontend validates input
          ↓
Backend receives request
          ↓
Backend validates transaction
```

```
          ↓
Fraud Detection Service
          ↓
Risk score generated
          ↓
Backend stores result
          ↓
PostgreSQL
          ↓
Alert generated if risk is high
          ↓
Dashboard displays result
```

Component Interaction

Component	Interacts With	Purpose
Frontend	Backend	Sends requests and displays results
Backend	PostgreSQL	Stores and retrieves transaction data
Backend	Fraud Service	Requests fraud analysis
Fraud Service	Backend	Returns risk score/result
PostgreSQL	Backend	Provides persistent storage
Docker Compose	All containers	Manages services and networking

Quality Attribute Analysis

Security – Authentication and authorization protect system access. Sensitive configuration values should be stored as environment variables or secure secrets.

Scalability – The frontend, backend and fraud detection services are separated into containers, allowing individual services to be scaled independently.

Performance – The backend handles API requests while the fraud detection service performs analysis separately. This separation prevents all processing from being placed in one component.

Reliability – Database persistence through Docker volumes helps protect stored transaction data. Service health checks and proper error handling can further improve reliability.

Maintainability – The system is divided into frontend, backend, fraud detection and database components. A change in one component can therefore be made without rewriting the complete system.

Availability – Containerized services can be restarted independently. In a production environment, multiple instances and health monitoring can further improve availability.

Architectural Justification

Why Layered/Service-Based Architecture?

The selected architecture separates the system into clear responsibilities:

```
Presentation Layer
       ↓
Application/API Layer
       ↓
Fraud Detection Service
       ↓
Data Layer
```

This is suitable because the project has different responsibilities such as user interaction, transaction processing, fraud analysis and data storage.

Advantages

Modularity – Each major component performs a specific responsibility.

Scalability – Individual services can be scaled when the workload increases.

Maintainability – Developers can modify one component without affecting the complete system.

Security – Security controls can be applied at the API and service levels.

Reliability – Failure in one service can be isolated and handled without necessarily stopping the entire system.

Portability – Docker allows the application to run consistently across different environments.

Final Conclusion

The proposed Fraud Detection and Financial Transaction Monitoring System provides a structured solution for monitoring financial transactions and identifying suspicious activities.

Git is used for version control, branching, collaboration, commit tracking and source-code management. Docker is used to containerize the frontend, backend, fraud detection service and database, providing consistency and portability.

The architecture separates presentation, application processing, fraud analysis and data storage. This improves security, scalability, performance, reliability and maintainability.

The combination of Git, Docker, Docker Compose and a modular software architecture provides a suitable foundation for developing, testing and deploying the proposed full-stack application.

IMAGE 3: DOCKER COMPOSE ARCHITECTURE DIAGRAM

This diagram shows the Docker Compose services, networking, volumes, and communication between containers.